

What This Strategy Actually Does

A code-level analysis of the *AI Hedge Fund Bot*

Repository `AI_Brain_Correlation_Hedging` · commit `368355b` · branch `main`
Stack: FastAPI · MetaTrader 5 · DeepSeek `deepseek-chat` · SQLite
Generated 8 September 2026 — derived entirely from source, not from documentation

Executive summary

This is an **LLM-discretionary, multi-timeframe, fixed-risk directional bot**. Every 30 seconds it walks a list of four instruments. For each one it pulls 10 Daily and 24 H1 candles from MetaTrader 5, computes a set of mechanical features, and asks DeepSeek to act as a discretionary trader and return `BUY`, `SELL` or `HOLD` with a 0–100 conviction score. If the answer is actionable, it immediately sends a fixed 0.01-lot market order with a stop 0.20% away and a target 0.40% away. It then never touches the position again — the broker's stop-loss or take-profit is the only exit. Closed trades are settled from MT5 deal history and distilled into short text "experiences" that are pasted into future prompts.

There is no back-tested rule set underneath any of this. **The entire edge is the language model's judgement** over roughly 34 candles of price data, restated fresh every 30 seconds with no memory of its own previous answers beyond the handful of closed-trade summaries the system retrieves for it.

The repository name describes a strategy that does not exist in the code.

The folder is called *AI Brain **Correlation Hedging***. Nothing in the codebase measures correlation between instruments, and nothing opens an offsetting position to neutralise risk. Each of the four symbols is analysed by a completely independent LLM call that is never told what the other three are doing. The word "hedge" appears only in the product name "*AI Hedge Fund*" on the dashboard. See §12 for the one place where offsetting positions *can* arise — accidentally, and without any of the properties a hedge is supposed to have.

1. Where the logic lives

File	Responsibility in the strategy
<code>core/engine.py</code>	The per-cycle pipeline, the risk gate, the main loop
<code>core/runtime.py</code>	Shared state, MT5 connection, single-writer lock, restart recovery
<code>core/reconciler.py</code>	Reads MT5 and corrects the database; produces P&L and experiences
<code>ai/brain.py</code>	The DeepSeek call, response validation, safe-HOLD fallbacks
<code>ai/prompts.py</code>	The actual trading instructions given to the model
<code>ai/memory.py</code>	Distilling closed trades and retrieving them into later prompts
<code>market/data_engine.py</code>	Candle fetch and all mechanical feature computation
<code>market/execution.py</code>	Order construction: price, SL, TP, volume, idempotency tag
<code>market/news.py</code>	Extension point; the default provider returns <code>None</code>
<code>config.py</code>	Every tunable parameter, loaded from <code>.env</code>

2. The cycle

`trading_loop()` runs continuously in one asyncio task, started once at process boot. The dashboard's Start/Stop button only flips a boolean the loop reads — it can never spawn a second loop, so a browser refresh cannot duplicate an order.

```
every 30s, while is_running:
    | heartbeat the single-writer engine lock
    | ensure MT5 is connected          → if not: skip cycle, log ERROR
    | RECONCILE (3 passes, read-only against MT5)    ...$9
    |
    | for each symbol in [EURUSDm, GBPUSDm, BTCUSDm, XAUUSDm]: ← strictly sequential
        | 1. fetch 10×D1 + 24×H1 candles, tick, account snapshot
        | 2. compute mechanical features          ...$5
        | 3. persist market_state + (throttled) equity snapshot
        | 4. retrieve up to 5 past experiences      ...$10
        | 5. ask DeepSeek → BUY / SELL / HOLD + score 0-100    ...$4
        | 6. risk gate → final_decision (+ override_reason)    ...$6
        | 7. persist decision row (valid or not)
        | 8. if BUY/SELL: write PENDING row → order_send → update row    ...$7,$8
```

Symbols are processed **one after another, not in parallel**, and each is a blocking HTTP round-trip to DeepSeek (30 s timeout). If the model is slow, one cycle can overrun the 30 s interval — the loop sleeps *after* the work, so the real cadence is `work + 30s`, not a fixed 30 s.

3. Instrument universe and cadence

Item	Value	Consequence
Symbols	EURUSDm, GBPUSDm, BTCUSDm, XAUUSDm	Two majors, one crypto, one metal — the <code>m</code> suffix indicates Exness-style micro accounts
Timeframes	D1 (macro) + H1 (micro), labelled D1+H1	Only 10 daily and 24 hourly bars are ever seen
Interval	30 s default; UI offers only 30 s or 1 hr	At 30 s the bot re-decides on H1 data ~120× per candle
AI calls	4 per cycle	≈ 11,500 DeepSeek calls/day at 30 s — a real recurring cost

Decision frequency is wildly out of scale with the data.

The bot asks for a fresh decision every 30 seconds, but the fastest input it has is an **H1 candle**. Between two consecutive decisions, 119 times out of 120 the candle data is *identical* — only the live tick has moved. Any change in the answer across those cycles is model non-determinism rather than new information. Because there is no cap on positions by default (§6), that same near-identical view can be acted on repeatedly.

4. The signal: what the model is actually asked

The system prompt (`ai/prompts.py`) casts the model as "an elite quantitative trader and market analyst" and instructs it to analyse price action, market structure, trend, momentum, support and resistance, higher-highs/lower-lows, and potential breakouts or reversals — with Daily as the macro trend and H1 as micro momentum. It must decide "based ONLY on the supplied market data" and reply in strict JSON:

```
{
  "daily_trend":      "bullish" | "bearish" | "neutral",
  "h1_trend":        "bullish" | "bearish" | "neutral",
  "momentum":        "strong" | "moderate" | "weak" | "neutral",
  "market_structure": "higher_highs" | "lower_lows" | "ranging" | "expanding" | "compressing",
  "market_regime":    "trending" | "ranging" | "choppy" | "volatile",
  "setup":            "breakout" | "pullback" | "reversal" | "range_fade" | ...,
  "score":            0-100,          ← conviction, explicitly NOT a probability
  "signal":           "BUY" | "SELL" | "HOLD",
  "daily_analysis":   "1 sentence",
  "h1_analysis":      "1 sentence",
  "reasoning":        "exactly 2 sentences"
}
```

What is in the user message

- 1. The 10 Daily candles as raw CSV (time, open, high, low, close, tick_volume, spread, real_volume).
- 2. The 24 H1 candles as raw CSV.
- 3. The engine's own computed features, labelled "measured, not opinion", with the explicit note: "These are mechanical indicators. You may disagree with them; state why in your reasoning if you do."
- 4. Up to 5 retrieved past trades, if any exist (§10).
- 5. News context — currently always absent.

On a fresh database with news disabled, the assembled prompt is byte-identical to the original two-CSV prompt. Sections are appended only when they have content.

How the answer is policed

`brain.py::get_ai_decision()` is built so it **can never raise into the trading loop**. Every failure path — missing API key, timeout, HTTP error, non-JSON body, unexpected envelope, malformed JSON, invalid signal, out-of-range score, empty reasoning — returns a well-formed dict with `signal: "HOLD"` and a status of `error` or `invalid`. The failure is still written to the `decisions` table so it surfaces in the dashboard rather than vanishing.

Field	Accepted	Rejected → safe HOLD
signal	BUY / SELL / HOLD, any case; alias key <code>decision</code>	Anything else, or non-string
score	int, integral float, or numeric string, 0-100; alias <code>confidence_score</code>	Out of range, non-integral, bool
reasoning	Any non-empty string; alias <code>logic</code>	Missing or blank
The six label fields	Lower-cased free text — <i>not</i> constrained to the enum	Never rejected; a creative label cannot block a trade

Temperature is `0.1` and `response_format` is `json_object`, so the model is pushed toward determinism and valid JSON. Note that the schema is enforced by the bot, not by the API — DeepSeek's JSON mode guarantees syntax, not these keys.

5. Engine-computed features

These are deterministic measurements from `data_engine.py`. They serve three purposes: context for the model, stored provenance in `market_states`, and slicing dimensions for both analytics and memory retrieval. **They never generate a signal by themselves.**

Feature	Formula as implemented
<code>atr_h1</code>	Mean of the last 14 True Ranges on H1 (falls back to the mean of whatever exists).
<code>atr_pct</code>	<code>atr_h1 / last_close × 100</code>
<code>volatility_bucket</code>	Latest true range vs the 25th/75th nearest-rank percentile of the same 24-bar window → <code>low</code> / <code>normal</code> / <code>high</code> . Self-relative, so it means the same thing for EURUSD and BTC.
<code>daily_trend</code> <code>h1_trend</code>	First-to-last close % change vs a threshold of $(\text{range} \div \text{first} \times 100) \times 0.20$ → <code>bullish</code> / <code>bearish</code> / <code>neutral</code> . The threshold scales with the instrument's own range rather than being a fixed number of pips.
<code>momentum</code>	Mean of last 3 closes vs prior 3, divided by ATR% → <code>strong_bullish</code> (>1.0), <code>bullish</code> (>0.3), <code>bearish</code> (<-0.3), <code>strong_bearish</code> (<-1.0), else <code>neutral</code> .
<code>market_structure</code>	Max/min of the recent half vs the earlier half of the 24 H1 bars → <code>higher_highs</code> / <code>lower_lows</code> / <code>expanding</code> / <code>compressing</code> .
<code>market_regime</code>	Efficiency ratio $ \text{close}_{24} - \text{close}_1 \div \sum \text{close}_i - \text{close}_{i-1} $ → <code>trending</code> (>0.45), <code>ranging</code> (<0.20), else <code>choppy</code> .
<code>session</code>	UTC hour of the last H1 bar → <code>asian</code> / <code>london</code> / <code>london_ny_overlap</code> / <code>new_york</code> / <code>off_hours</code> .
<code>spread</code> , <code>spread_pct</code>	<code>ask - bid</code> , raw and as % of last close. Measured but never used as a gate.

Two implementation details worth knowing.

(a) `_market_regime(daily_df, hourly_df)` accepts the daily frame but **never reads it** — the regime is computed purely from the 24 H1 closes, despite being the label that drives memory retrieval and analytics slicing.

(b) Candles are fetched with `copy_rates_from_pos(..., 0, n)`, and position 0 in MT5 is the *current, still-forming* bar. So the newest Daily candle the model sees is a partial bar whose "close" is just the current price, and every trend, structure and momentum label is computed with that incomplete bar included.

6. The risk gate

`apply_risk_checks()` converts the AI's signal into what the engine will actually do, returning `(final_decision, override_reason)`. Checks are evaluated in order:

#	Check	Result	Default
1	AI response status is not <code>ok</code>	Force HOLD — an unusable response can never become an order	always on
2	Signal is not BUY/SELL	Pass through unchanged	always on
3	MT5 not connected	Force HOLD	always on
4	<code>ai_score < MIN_SCORE_TO_TRADE</code>	Force HOLD	OFF (0)
5	Open positions on this symbol ≥ <code>MAX_OPEN_POSITIONS_PER_SYMBOL</code>	Force HOLD	OFF (0)

Overrides are never silent: the decision row stores `final_decision` alongside `ai_signal` plus a human-readable `override_reason`, a `WARN / OVERRIDE` event is written, and the dashboard shows an amber banner.

With stock settings, the only real filters are "the AI replied" and "MT5 is up".

Both discretionary gates ship **disabled** — deliberately, to preserve the original author's trading behaviour. That means *every* valid BUY or SELL becomes a new position, on every symbol, on every cycle, with no cap. At 30 s over four symbols that is a theoretical maximum of **8 new positions per minute**, each 0.01 lots, accumulating indefinitely. A score of 51 and a score of 99 are treated identically.

7. Order construction and the trade math

From `execution.py::execute_trade()`. The author notes the price/SL/TP/volume maths is unchanged from the original code.

Parameter	Value	Notes
Order type	Market (<code>TRADE_ACTION_DEAL</code>)	BUY fills at ask, SELL at bid
Volume	<code>0.01</code> lots, constant	No sizing from equity, ATR, or conviction score
Stop-loss	0.20% of entry	BUY <code>price×(1-0.002)</code> · SELL <code>price×(1+0.002)</code>
Take-profit	0.40% of entry	BUY <code>price×(1+0.004)</code> · SELL <code>price×(1-0.004)</code>
Rounding	To the broker's <code>symbol_info.digits</code>	Correctly handled per instrument
Deviation	20 points	Max slippage tolerated on the fill
Filling mode	<code>ORDER_FILLING_IOC</code> , hardcoded	See risk note below
Magic number	<code>100001</code>	Marks the bot's own positions; manual trades are left alone
Comment tag	<code>AIQ-<12 hex></code> (16 chars)	Kept short because MT5 truncates comments at ~31 chars

The number that defines this strategy

Every trade is a fixed **1 : 2 risk-to-reward**. That fixes the break-even hit rate before costs:

```
break-even win rate = risk ÷ (risk + reward) = 0.20 ÷ (0.20 + 0.40) = 33.3%
```

So the LLM must be right more than one time in three, on a 34-candle view, for the system to be flat before spread, commission and swap. Including costs the true hurdle is higher — and because the stop is a fixed 0.20% while spreads are not, the cost drag differs enormously per instrument. A 1-pip spread against a 22-pip stop on EURUSD is ~4.5% of risk; a wide crypto or metal spread during thin hours can be a double-digit percentage of the same 0.20% stop.

The same percentage on four very different instruments

0.20% is applied identically to a major FX pair, gold, and Bitcoin — instruments whose hourly ATRs differ by more than an order of magnitude in percentage terms. ATR *is* computed and handed to the model, but it is never used to size the stop. In practice this means the stop sits at a different multiple of ATR on every symbol: comfortably wide on EURUSD, and potentially inside routine noise on BTCUSD, where a 0.20% move can occur within minutes. The result is that the effective strategy — how long trades last and how often stops are grazed — is quietly different per symbol, without that ever being an explicit decision.

Risk: the filling mode is hardcoded and the comment contradicts the code.

`execution.py:261-262` reads `// Let MT5 use the symbol's supported filling mode immediately above "type_filling": mt5.ORDER_FILLING_IOC`. IOC is *not* universally supported — many brokers and symbols require FOK or RETURN. On such a symbol every order returns retcode `10030` (Unsupported filling mode) and the bot will log rejections forever while never opening a trade. The correct fix reads `symbol_info.filling_mode` and picks a supported value. This sits alongside the position-ticket risk the author already flagged as the second most likely thing to break on a live broker.

8. Order lifecycle

```
insert_trade_intent() → row written as PENDING *before* the order is sent
                        (carries the unique client_order_id)
    |
    ▼
order_send()
    |
    ├── retcode DONE / DONE_PARTIAL → EXECUTED + entry, SL, TP, tickets, INFO event
    ├── any other retcode           → REJECTED + retcode & broker comment persisted
    ├── raised exception            → FAILED   + reason persisted
    └── returned None (ambiguous)    → stays PENDING on purpose; the reconciler
                                    decides by ASKING MT5, never by re-sending
```

Writing the intent before the send is the whole trick: if the process dies between `order_send` and the database write, the orphaned PENDING row is what lets the reconciler find the real outcome in MT5's history instead of blindly firing a second order. Rejections used to raise and get swallowed by the loop's generic `except` ; they are now persisted as data.

9. Exits — the most important section

The bot has no code path that closes a position.

A grep for `order_send` across the entire backend returns exactly one call site: the single opening market order. There is no `TRADE_ACTION_SLTP` , no close request, no modify. The reconciler's own docstring states it "never writes to MT5 — it only reads MT5 and corrects our mirror." **Every exit is executed broker-side by the stop-loss or take-profit attached at entry.**

Concretely, the strategy has:

Exit mechanism	Present?
Broker-side fixed stop-loss / take-profit	Yes — the only exit
Trailing stop	No
Move to break-even	No
Partial take-profit / scaling out	No
Time-based exit (max holding period)	No
Exit when the AI reverses its view	No — see below
End-of-session / weekend flat	No
Portfolio drawdown kill-switch	No

What happens when the AI changes its mind

Suppose the model says BUY on EURUSD at cycle n and SELL at cycle $n+1$. The long is **not closed**. Instead a *new short* is opened alongside it. With `MAX_OPEN_POSITIONS_PER_SYMBOL=0` (the default) the account can therefore hold simultaneous longs and shorts on the same symbol, each with its own stop and target, both paying spread, and — on a netting account — behaving differently again.

This is the only "hedging" in the repository, and it is accidental.

Two opposing positions on one symbol do offset directionally, but this is not a hedge in any useful sense: it is unplanned, unsized, arises from model indecision rather than risk analysis, costs two spreads to enter and two to exit, and both legs can still be stopped out for a combined loss if price whipsaws through both stops. Given the repository is named for correlation hedging, it is worth being explicit that this is a side-effect of missing exit logic, not an implementation of the idea.

10. Reconciliation — how trades get settled

`core/reconciler.py` runs at startup and once at the top of every cycle. MT5 is the source of truth; the database is a mirror. Three isolated passes, so one MT5 hiccup cannot abort the others:

Pass	What it does
1. <code>resolve_pending_trades</code>	For PENDING rows older than 120 s, search MT5 deal history for the <code>AIQ-</code> comment tag. Found → adopt as EXECUTED with the real tickets and fill price. Not found → mark FAILED. This is what makes a restart safe.
2. <code>close_finished_trades</code>	For each open trade whose position MT5 no longer holds, aggregate every deal on that position: P&L <i>net of commission and swap</i> , exit price and close time all come from MT5 — never estimated. Sets CLOSED, classifies WIN/LOSS/BREAKEVEN, computes the realised R multiple against the original stop distance, and writes an experience row. If history isn't available yet it retries next cycle rather than guessing.
3. <code>adopt_untracked_positions</code>	Any live position with magic <code>100001</code> and no database row gets one, so it appears on the dashboard and counts toward the per-symbol cap.

Latency note: because reconciliation runs once per cycle, a position that hits its target is only *recorded* as closed up to one interval later (≤ 30 s, or ≤ 1 hour on the slow setting). The P&L is exact when it lands; only the bookkeeping — and therefore the experience that feeds back into the prompt — lags.

11. The learning loop

`ai/memory.py` is explicit that this is **retrieval-augmented adaptation, not training**. DeepSeek's weights are never touched and the model is stateless between calls. What persists is the bot's own record of what happened.

```
trade CLOSED → record_experience()
                symbol · regime · setup · direction · score bucket · volatility
                session · entry/exit · R multiple · P&L · outcome · holding minutes
                + a one-sentence "lesson", deliberately free of causal claims
                |
                ▼
next decision → retrieve_relevant() progressively broader filters:
                symbol + regime + volatility
                → symbol + regime → symbol → regime → anything
                | (max 5 rows)
                ▼
                build_prompt_context()
                "=== YOUR PAST TRADES IN COMPARABLE CONDITIONS ==="
                n wins / m losses, average R
                "This is a small sample and is NOT proof of an edge."
```

The intellectual honesty here is notable — the prompt itself warns the model not to over-read the sample, and the lesson text avoids claiming *why* a trade worked. Two caveats follow from the implementation:

- **"Relevant" means "recent".** `find_experiences` orders by `id DESC` and takes the first 5. Within a filter bucket there is no similarity ranking — just recency.
- **The fallback tier is very wide.** The last attempt is an unfiltered query. Early on, when few trades have closed, the model can be shown trades from a *different symbol in a different regime* under the heading "comparable conditions". The header is only accurate once the database is well populated.

On a fresh database this block is omitted entirely, so memory has zero influence until real trades have closed. It never influences *position sizing* or the gates — only the model's reading of the setup.

12. Strategy classification

Dimension	This system
Style	Discretionary, outsourced to an LLM — not systematic in the usual sense
Direction	Long/short, symmetric
Holding period	Undefined by design; ends whenever a 0.20%/0.40% band is touched. Effectively intraday scalping-to-swing depending on instrument volatility
Edge thesis	That an LLM reading 34 candles plus 8 mechanical features can pick direction >33.3% of the time at 1:2 R:R
Sizing	Fixed 0.01 lots — no Kelly, no volatility targeting, no equity scaling
Risk model	Per-trade only, via the stop. No portfolio layer whatsoever
Market exposure	Unbounded and uncontrolled — grows with every actionable signal
Validation	None. No backtest, no walk-forward, no paper-trading record in the repo

The concentration problem the repository name gestures at.

EURUSD and GBPUSD are typically 0.85–0.95 correlated; gold and Bitcoin both trade substantially as dollar-and-risk-sentiment proxies. Four independent LLM calls, each shown a chart that is being driven by the *same* underlying dollar move, will frequently reach the *same* conclusion. The result is not four diversified positions — it is one concentrated bet against the dollar, taken four times over, with roughly four times the intended risk and none of it visible to any part of the system. Nothing in the code measures this, and no prompt tells any symbol's analysis what the other three just decided. This is precisely the gap that "correlation hedging" would close.

13. Traceability and persistence

The system's real engineering strength is its audit chain. Every executed order can be followed end to end:

```
market_states → decisions (ai_signal, features, raw model response, latency)
                |
                |— decisions (final_decision + override_reason) ← nothing overrides silently
                ▼
            trades (client_order_id, PENDING)
                |— trades (EXECUTED / REJECTED + MT5 retcode & tickets)
                |— trades (CLOSED + P&L, R multiple, commission, swap)
                ▼
            experiences → fed back into the next prompt
```

Seven SQLite tables, WAL mode so the dashboard reads while the loop writes, ISO-8601 UTC strings throughout, and a per-thread connection with a Python-side write lock. Only raw data is stored — no rendered charts — so the dashboard reconstructs the equity curve from points every time and the numbers can never drift from the underlying rows. Analytics (`database/analytics.py`) computes P&L, win rate, profit factor, expectancy, average R, max drawdown and streaks on demand, and attaches a `sample_note` that explicitly says the figures are "not a statistically meaningful sample" below 30 closed trades.

14. Structural gaps, ranked

#	Gap	Why it matters
1	No position cap by default	Exposure compounds without limit; a persistent view becomes an oversized position by accretion rather than decision
2	No cross-symbol / correlation awareness	Four correlated longs read as diversification but are one leveraged bet (§12)
3	No portfolio risk limits	No daily loss cap, no drawdown kill-switch, no max total exposure
4	Filling mode hardcoded to IOC	Can reject 100% of orders on brokers/symbols requiring FOK (§7)
5	No exit on view reversal	Produces unintended opposing positions instead of flattening (§9)
6	Stops not volatility-scaled	0.20% means something different on EURUSD vs BTC; ATR is computed but unused for sizing
7	No spread or session gate	Both are measured and shown to the model, but neither can block a trade in thin or dislocated conditions
8	Partial current candle included	The newest D1 and H1 bars are still forming, so every label is computed over an incomplete bar (§5)
9	Decision rate » data rate	120 decisions per H1 candle; most re-decisions see identical data (§3)
10	Score is uncalibrated	Labelled conviction, not probability — and with the gate off, it currently affects nothing at all
11	<code>market_regime</code> ignores the daily frame	The label driving memory retrieval is H1-only despite its signature (§5)
12	Legacy <code>last_confidence</code> is overwritten per symbol	The global field ends each cycle holding whichever symbol was processed last; the per-symbol tabs are the trustworthy view

15. What is verified, and what is not

The engineering around the strategy is genuinely well tested — **against a fake broker**. `tests/fake_mt5.py` stands in for the Windows-only MetaTrader5 package, and two suites (48 recovery checks, 31 API checks) pass:

- Decisions, trades and equity survive a full process restart; the dashboard rebuilds from SQLite rather than opening blank.
- The engine does **not** auto-resume trading on restart (deliberate: a crash-loop must not fire orders unattended).
- A malformed AI response creates no trade and is recorded as an override to HOLD with a reason.
- An orphaned PENDING order is adopted from MT5 history and **no duplicate order is sent**.
- An order MT5 never received is marked FAILED rather than retried.
- Untracked positions carrying the magic number are adopted; rejections are persisted with a reason.
- Closing a trade produces P&L and exactly one experience; reconciliation is idempotent.
- Every real trade traces back to a decision; one latest decision is kept per symbol.

None of this has ever touched a live broker. A fake MT5 cannot reproduce requotes, partial fills, variable spreads, filling-mode restrictions, or the order-ticket vs position-id divergence the author already flagged. Equally important: *none of these tests say anything about whether the strategy makes money.* They verify that the plumbing is honest, not that the signal has an edge.

16. Parameter reference

Env var	Default	Effect on the strategy
SYMBOLS	EURUSDm,GBPUSDm,BTCUSDm,XAUUSDm	The universe; comma-separated override
SL_PERCENT	0.002	Stop distance — the denominator of every R multiple
TP_PERCENT	0.004	Target distance; with SL fixes the 33.3% break-even hit rate
LOT_SIZE	0.01	Constant position size
MAGIC_NUMBER	100001	Ownership tag for reconciliation
DEVIATION	20	Max slippage in points
MAX_OPEN_POSITIONS_PER_SYMBOL	0	0 = unlimited stacking. Set to 1 to make the strategy one-position-per-symbol
MIN_SCORE_TO_TRADE	0	0 = the score is ignored. Set to e.g. 70 to make conviction matter
DEFAULT_INTERVAL	30	Seconds between cycles; persists across restarts
DEEPSEEK_MODEL / TEMPERATURE / TIMEOUT	deepseek-chat / 0.1 / 30	The signal source itself
MEMORY_ENABLED / MEMORY_MAX_EXPERIENCES	true / 5	Experience retrieval into the prompt
EQUITY_SNAPSHOT_MIN_SECONDS	60	Throttle so a 4-symbol cycle writes one equity point, not four
RESUME_ENGINE_ON_STARTUP	false	Whether a restart resumes trading (off = safe)
PENDING_TRADE_TIMEOUT_SECONDS	120	Age at which a PENDING row is treated as orphaned
NEWS_ENABLED / NEWS_PROVIDER	false / null	Extension point only; no provider implemented
DB_PATH	data/quantbot.db	Data lives outside backend/ so redeploying code cannot touch history

17. Bottom line

Two very different pieces of work are sitting in this repository. The **infrastructure** — idempotent execution, crash-safe reconciliation, full decision-to-P&L traceability, honest analytics that refuse to overclaim on small samples — is careful, well-reasoned and properly tested. The **strategy** it carries is a single unvalidated hypothesis: that a general-purpose language model, shown 34 candles and eight indicators, can call direction better than one in three at a fixed 1:2 payoff, on four instruments at once, with no position limit, no portfolio risk layer, and no exit logic beyond a fixed percentage bracket.

The first thing worth doing is not more code. It is turning on `MAX_OPEN_POSITIONS_PER_SYMBOL=1` and `MIN_SCORE_TO_TRADE`, running on a demo account, and letting `/api/analytics/breakdown/ai_score` answer the only question that matters — whether the model's conviction score has any relationship at all to outcomes. Everything else, including the correlation and hedging logic the repository is named after, is downstream of that answer.

Analysis derived from: `backend/core/{engine, runtime, reconciler}.py` · `backend/ai/{brain, prompts, memory}.py` · `backend/market/{data_engine, execution, news}.py` · `backend/database/{schema.sql, connection.py, repository.py, repo_trading.py, repo_context.py, analytics.py}` · `backend/{config, main}.py` · `frontend/templates/index.html` · `tests/{fake_mt5, test_restart_recovery, test_api}.py` · `task_pending/README.md`